TRI (Transformative Resource Intelligence) - Master Project Blueprint

This document serves as the comprehensive master blueprint for the TRI project. It is meticulously optimized for the Innobot 2 Hackathon (Smart Living Domain), hosted by the IEEE Robotics & Automation Society (RAS).

The system design prioritizes the exact Innobot scoring rubric:

- **Technical Ownership (30 points):** Clear separation of AI vs. hard-coded deterministic logic.
- **Automation Impact (25 points):** Visible, closed-loop robotic-style workflows.
- **Innovation & AI Depth (20 points):** Tagged RAG architecture and explainable decision engines.
- **Code + UI (15 points):** Audit-ready repository and transparency-focused UI.
- **Pitch (10 points):** A deterministic, 7-minute risk-free demo flow.

Section 1: Project Overview & Vision

Hackathon: Innobot 2 **Domain:** Smart Living **Project Name:** TRI – Autonomous Academic Automation Agent

1.1 The Problem Statement

Students currently navigate a highly fragmented academic ecosystem. They rely on scattered sources to collect semester-specific notes and previous-year questions (PYQs). This unstructured process causes cognitive overload and inefficient time management. Existing platforms act as passive storage drives rather than active intelligence engines.

1.2 The TRI Solution & Positioning

TRI is an **autonomous academic decision-making system**. TRI continuously automates academic preparation by sensing learning gaps, reasoning over knowledge resources, and acting through adaptive planning without manual intervention.

Final Positioning Statement: > "TRI is an autonomous academic automation system that senses learning gaps, reasons over structured knowledge using RAG, and dynamically adapts preparation strategies through a closed feedback loop."

1.3 The Automation Loop

TRI abandons the "EdTech App" model and operates on a robotics-inspired `Sense → Analyze → Decide → Act → Adapt` automation cycle.

TRI Automation Cycle Flow:

```
[ Upload Syllabus ]  (Sense)
        ↓
[ TRI Detects Topics ]  (Analyze)
        ↓
[ Auto Resource Mapping ]  (Analyze)
        ↓
[ Auto Study Planning ]  (Decide)
        ↓
[ Auto Evaluation ]  (Act)
        ↓
[ Weakness Detection ]  (Adapt)
        ↓
[ Plan Reconfiguration ]  (Adapt)
```

Section 2: Core Architecture & The 5 Engines

To ensure a highly stable, deterministically orchestrated system that passes strict technical audits, TRI's scope is frozen to 5 core execution engines. **We do not claim full web autonomy, large-scale crawling, or real-time learning analytics.** The architecture is contained, understandable, and explainable.

2.1 Core Technology Stack

- **Frontend Ecosystem:** Next.js 14 (App Router), React, TypeScript, Tailwind CSS.
- **Backend Application Server:** Python Flask REST API.
- **AI Engine:** Google Gemini (Pro models for generative text, `text-embedding-004` for vector embeddings).
- **Vector Database:** FAISS (CPU-optimized) for localized vector similarity search.
- **Relational Database:** SQLite to map and serve structured metadata.

2.2 The 5 Core Engines

The TRI Orchestrator (manually written deterministic Python logic) routes interactions across these 5 engines:

1. **Document Intelligence (RAG Engine):** Extracts, chunks, tags, and embeds PDF syllabus data into FAISS.
2. **Resource Mapper:** A deterministic script that searches strictly whitelisted educational domains (`.edu`, `GeeksforGeeks`, `Wikipedia`) to fill detected syllabus gaps.
3. **Adaptive Planner:** Calculates priority mathematically based on quiz scores and uses AI to generate actionable daily schedules.
4. **Evaluation Engine:** Grades user input, provides feedback, and updates the student's learning state memory in SQLite.
5. **Context Assistant:** The doubt-resolution interface powered entirely by Tagged RAG retrieval to prevent hallucination.

Section 3: The Knowledge Engine, Tagged RAG & Resource Mapper

TRI employs **metadata-aware tagged retrieval**, enabling subject-specific and task-aware contextual reasoning.

3.1 Tagged RAG Pipeline

When a document enters TRI:

1. **Chunking:** Text is split into chunks of 400 to 600 tokens (with a 50-token overlap).
2. **AI Metadata Tagging:** Gemini automatically analyzes the chunk and attaches structural JSON metadata (Subject, Topic, Exam Importance).
3. **Storage:** Embedded via `text-embedding-004` and stored in FAISS, with the metadata linked in the SQLite database.

Metadata Schema Example:

```
{
  "chunk_id": "dbms_norm_023",
  "content": "Normalization reduces redundancy...",
  "metadata": {
    "subject": "DBMS",
    "topic": "Normalization",
    "resource_type": "Article",
    "exam_relevance": "High",
    "ingestion_type": "RAG_PROCESSED"
  }
}
```

3.2 Context-Aware Retrieval Modes

TRI builds a context filter *before* searching FAISS.

- **Doubt Mode:** Filters by `subject` + `topic`.
- **Planner Mode:** Filters by `subject` + `exam_relevance=High`.
- **Quiz Mode:** Filters by `topic` + `conceptual_chunks`.

3.3 Autonomous Web Crawler & Validation Layer

If the syllabus mentions a topic but lacks detailed notes, the crawler searches whitelisted domains. It then makes a strict routing decision:

- **Route A: Download & RAG (Text-Heavy Content):**
  - **Trigger:** Finds a text tutorial on GeeksforGeeks.
  - **Action:** Scrapes, cleans, chunks, tags, and embeds into FAISS. Database marks it as `RAG_PROCESSED`.
- **Route B: Store as Link (Media & Heavy Assets):**
  - **Trigger:** Finds a YouTube video lecture.

- **Action:** Bypasses scraping. Extracts URL/Thumbnail and stores in SQLite as `LINK_ONLY`. Renders as a clickable card in the UI.

Section 4: Relational Database Schema & Learning Memory

The backend uses these 5 core tables to synchronize the RAG vectors with the student's behavioral memory.

1. Table: `subjects`

- `id` (PK), `name` (String), `confidence_level` (Integer: 1 to 5)

2. Table: `topics`

- `id` (PK), `subject_id` (FK), `topic_name` (String), `priority` (String), `status` (String: Pending/Mastered/Weak)

3. Table: `mapped_resources` **(The Knowledge Router)**

- `id` (PK), `topic_id` (FK), `title` (String), `url` (String), `resource_type` (String), `ingestion_status` (String: RAG_PROCESSED or LINK_ONLY)

4. Table: `quiz_evaluations`

- `id` (PK), `topic_id` (FK), `score` (Integer: 0 to 10), `ai_feedback` (Text), `timestamp` (DateTime)

5. Table: `learning_state` **(Student Behavior Memory)**

- `topic_id` (FK), `attempt_count` (Integer), `last_score` (Integer), `revision_priority` (Float), `forgetting_index` (Float)

Section 5: Application Modules & RAS-Friendly Autonomy Features

5.1 Explainability Feature (Technical Ownership)

To prove the AI is making reasoned decisions, TRI explicitly justifies its actions. When a topic is marked as "Weak," the UI displays: **Why TRI marked this weak:**

- Low quiz score (4 / 10)
- High exam frequency (Appeared in 3 PYQs)
- Low revision count (0 revisions)

5.2 Autonomous Learning Mode (RAS-Friendly)

TRI features autonomous behavioral shifts.

- **Trigger:** The system detects the Exam is < 5 days away.
- **Action:** TRI automatically switches from "Learning Mode" to "Revision Mode." It halts the introduction of new topics and exclusively reschedules high-frequency PYQ topics.

5.3 The Hybrid Adaptive Study Planner

The Planner does not hallucinate. It uses a mathematical model:

1. **Rule Engine:** Calculates priority: `revision_priority × weakness_score × exam_proximity`.
2. **AI Optimization:** Gemini takes this matrix and refines it into a 7-day to 14-day human-readable schedule.

Section 6: UI/UX Design System & Transparency

Design Philosophy: A productivity-focused academic workspace. **Colors:** Background `#FAF3E1`, Surface `#F5E7C6`, Accent `#FF6D1F`, Text `#222222`.

6.1 Global Layout Structure (12-Column Grid)

- **Top Navigation (Fixed):** Logo, Breadcrumbs, User Profile.
- **Left Sidebar (3 Columns):** Context Panel (Active Subjects, Upload buttons).
- **Center Workspace (6 Columns):** Main functional area (Topic Reader, Quiz, Planner).
- **Right Insights Panel (3 Columns):** System Activity Terminal, Coverage Meters, Weak Area Alerts.
- **Bottom Dock:** Focus session timer and quick actions.

6.2 The 5 Core Screens Detailed

1. **Dashboard (Home):** Exam countdown, AI Readiness Score (e.g., 68%), Today's Tasks, Weak Topic Alerts.
2. **Subject Overview:** Unit list, topic progress bars, priority tags (High/Medium/Low).
3. **Topic Workspace:** Distraction-free area showing AI Notes (FAISS), External Links (`LINK_ONLY`), and Doubt Chat.
4. **Study Planner:** Calendar view with dynamic daily hour allocations.
5. **Practice / Quiz:** Minimal testing interface generating 5 to 10 questions with instant AI feedback.

6.3 System Activity Panel (Visible Automation)

To ensure judges visually *see* the background automation working, the UI features a live terminal:

```
[System Activity]
✓ Parsing syllabus structure...
✓ Extracting 42 topics...
✓ Mapping external resources from .edu...
✓ Updating study strategy...
✓ Detecting weak topics...
```

Section 7: Master Implementation Prompts & APIs

7.1 Backend System Prompt (The AI Brain)

Provide this to the Gemini initialization in Flask:

> "Act as TRI, an autonomous AI Academic Mentor. You have access to a FAISS vector database containing the student's syllabus and verified web-scraped context. Analyze student academic data, syllabus structure, and performance marks. Detect weak concepts and subjects. Explain WHY performance is low. Generate personalized academic notes grounded strictly in the provided RAG context. Create an adaptive study plan with daily targets. Output your response in a structured JSON format containing: Important Topics, Strong Subjects, Weak Subjects, Reason for Weakness, Personalized Notes, Study Plan, and Improvement Strategy."

7.2 Flask API Endpoints

- `POST /api/v1/upload`: PDF ingestion, chunking, Gemini auto-tagging, and FAISS embedding.
- `GET /api/v1/topic/<id>`: Returns syllabus data, filtered RAG notes, and `LINK_ONLY` URLs.
- `POST /api/v1/evaluate`: Grades answers out of 10, updates SQLite `learning_state`, and triggers the Orchestrator to adapt the planner.

Section 8: Audit-Ready GitHub Structure & AI Disclosure

To score maximum points on Technical Ownership (30 points), the repository is structured to cleanly separate concerns.

8.1 Repository Structure

```
TRI/
├── frontend/             # Next.js 14 UI
├── backend/              # Flask API & Orchestrator
├── rag/                  # FAISS & Tagging Logic
├── planner/              # Hybrid Planning Engine
├── evaluation/           # Grading Logic
├── docs/
│   ├── architecture.md
│   └── AI_USAGE.md       # Mandatory AI Disclosure
└── README.md
```

8.2 AI_USAGE.md (Disclosure Statement)

- **Google Gemini is utilized strictly for:** Embedding generation, unstructured text summarization, auto-tagging metadata, and evaluation grading feedback.
- **Manual Orchestration:** The `Sense → Analyze → Decide → Act → Adapt` loop, the Hybrid Planner mathematical weighting, the deterministic Resource Mapper filtering, and the FAISS context-routing logic are 100% deterministically coded in Python by the team.

Section 9: The 7-Minute Demo Execution Plan

This demo flow is highly optimized to fit strictly within the 7-minute limit while proving the complete automation loop. **Do not demo edge cases; stick to this deterministic path.**

1. **Upload Syllabus (Sense):** Upload a PDF. Point immediately to the System Activity Panel to show the automation parsing 42 topics and mapping resources in real-time.
2. **Open Topic (Analyze):** Show the topic workspace. Point out the AI-generated notes (from FAISS) and the mapped resources pulled deterministically from whitelisted sites.
3. **Ask Doubt (Act):** Use the Context Assistant to ask a specific question. Show how the Tagged RAG retrieves the exact answer without hallucinating.
4. **Take Quiz & Fail (Evaluate):** Intentionally score a 3 out of 10 on a practice quiz.
5. **Planner Auto-Updates (Adapt):** Navigate back to the Dashboard. Show the Explainability Feature explicitly stating *why* the topic is now flagged as "Weak," and show the Adaptive